Memory Management in Operating System

The term memory can be defined as a collection of data in a specific format. It is used to store instructions and process data. The memory comprises a large array or group of words or bytes, each with its own location.

The primary purpose of a computer system is to execute programs. These programs, along with the information they access, should be in the main memory during execution. The CPU fetches instructions from memory according to the value of the program counter.

To achieve a degree of multiprogramming and proper utilization of memory, memory management is important.

What is Main Memory?

Main memory, also known as primary memory or RAM (Random Access Memory), is the central storage area in a computer where the operating system, application programs, and current data are kept so they can be quickly reached by the device's processor.

It serves as the workspace for the computer's processor, storing data and instructions that are actively being used and executed.

Main memory is volatile, meaning it loses its contents when the computer is turned off.

There are two main types of RAM used in main memory:

- **Static RAM (SRAM):** Faster and more reliable, but also more expensive.
- **Dynamic RAM (DRAM):** Less expensive, but requires periodic refreshing to maintain data.

Main memory is directly accessible by the CPU, allowing for quick read and write operations, which is essential for efficient program execution.

What is Memory Management?

Memory management is a crucial function in a computer system that handles how memory (RAM) is allocated, used, and freed up for programs and data. It ensures that memory is efficiently used and prevents programs from interfering with each other.

The operating system (OS) is responsible for managing memory through a component called the **Memory Manager**.

Memory management is a method in the operating system to manage operations between main memory and disk during process execution. The main aim of memory management is to achieve efficient utilization of memory.

Why Memory Management is Required?

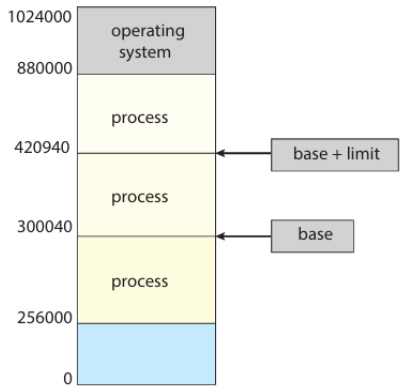
- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Base and Limit Register

Base and limit registers are fundamental components in computer systems for managing memory protection and process isolation.

The **base register** holds the starting physical address allocated to a process, while the **limit register** specifies the range or extent of the address space assigned to that process. This setup ensures that a process operates within its designated memory boundaries, preventing it from accessing or modifying memory allocated to other processes or the operating system.

When a process generates a logical address, the system adds this address to the base register's value to produce a physical address. The system then compares this physical address against the limit register to verify that it falls within the permitted range. If the address exceeds the limit, the system triggers a trap or exception, effectively preventing unauthorized memory access.



Address Binding

Address binding is the process by which an operating system maps logical addresses (used by programs) to physical addresses (actual locations in memory). This mapping ensures that a program's instructions and data are correctly placed in the computer's physical memory during execution.

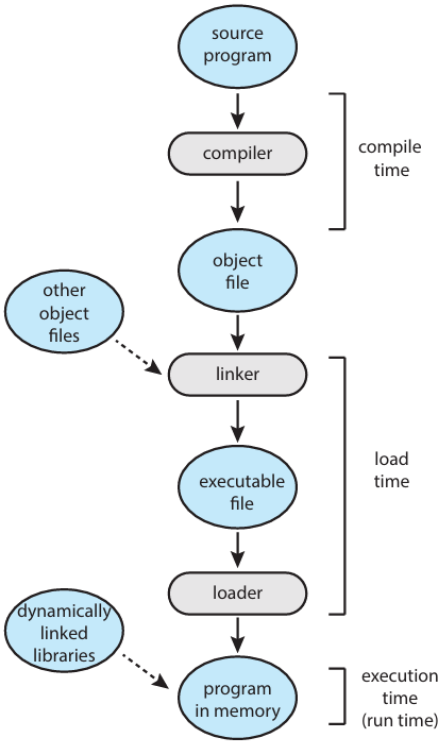


Figure 9.3 Multistep processing of a user program.

There are 3 primary types of address binding:

1. **Compile-time**: If you know where a process will be stored in memory when the program is compiled, then **absolute code** can be generated. For example, if you know that the process will reside at location R, then the compiler generated code will start at that location and continue from that point. However, if the starting location changes later, the code will need to be recompiled.
2. **Load time:** If it is not known at compile time where the process will reside in memory, then the compiler must generate **relocatable code.** The final

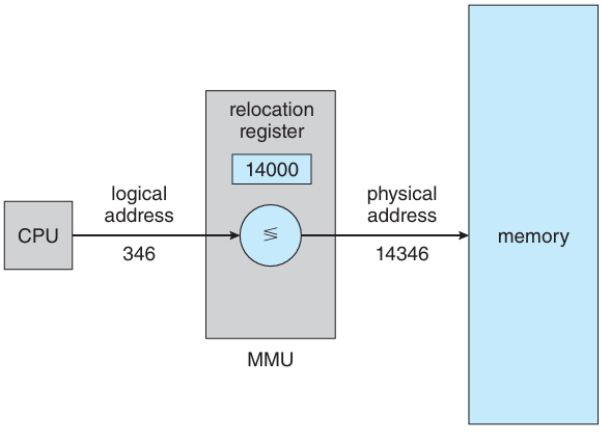
memory address is determined at load time. If the starting address changes, we only need to reload the code with the new address.

3. **Execution-Time (Dynamic)**: In this method, memory addresses are assigned during the program's execution. This allows for the most flexibility, as the program can be moved around in memory during its execution. However, it requires hardware support, such as a memory management unit (MMU), to translate logical addresses to physical addresses in real-time

Logical and Physical Address Space

Logical Address Space: An address generated by the CPU during program execution is known as a "**Logical Address**". It is also known as a Virtual address as it doesn't exist physically. Logical address space can be defined as the size of the process. A logical address can be changed. The collection of all logical addresses generated by a program is called the **logical address space**.

Physical Address Space: An address seen by the memory unit (i.e. the one loaded into the memory address register of the memory) is commonly known as a **"Physical Address"**. A Physical address is also known as a **Real address**. The set of all physical addresses corresponding to these logical addresses is known as **Physical address space.** A physical address is computed by MMU. The run-time mapping from virtual to physical addresses is done by a hardware device Memory Management Unit (MMU). The physical address always remains constant.



Memory Management Unit (MMU)

The **Memory Management Unit (MMU)** is a hardware component that manages virtual memory and caching functions. It is often part of the CPU or housed in a separate Integrated Chip (IC). The MMU handles data requests and determines whether to fetch the data from ROM or RAM.

Dynamic Loading

Loading is the process of **loading the program from secondary memory to the main memory** for execution.

With **Dynamic Loading** routines are only loaded when they are needed. Initially, the main program is loaded into memory, while other routines remain on disk in a relocatable format.

When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded. If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

Dynamic loading is useful for handling infrequent cases like error routines, reducing memory usage in large programs.

Dynamic Linking

Linking is the process of **connecting all the modules or the function of a program for program execution**.

What is Dynamic Linking? Linking is delayed until the program is running.

How It Works:

- A small piece of code, called a **stub**, is used to find the required library routine or load it if it's not already in memory.
- When the stub runs, it checks if the routine is already in memory. If not, it loads the routine into memory.

- The stub then replaces itself with the routine's address and executes the routine.
- On future calls, the routine is executed directly without any extra linking cost.

Why Use Dynamic Linking? It is especially helpful for managing libraries efficiently.

Role of the Operating System: The OS ensures the routine is in the process's memory space.

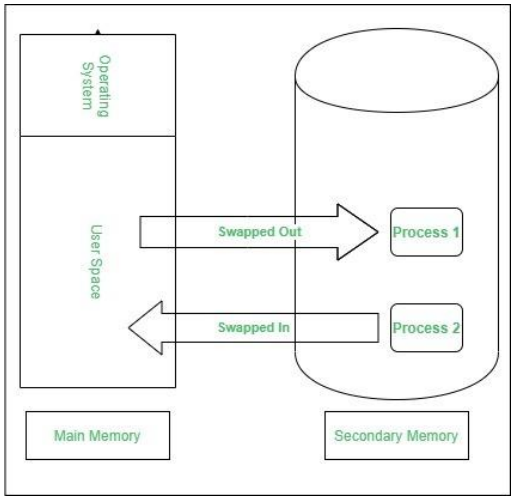
Swapping in Operating System

Swapping in an operating system is a **process that moves data or programs between the computer's main memory (RAM) and a secondary storage** (usually a hard disk or SSD). This helps manage the limited space in RAM and allows the system to run more programs than it could otherwise handle simultaneously.

It's important to remember that swapping is only used when data isn't available in RAM. Although the swapping process degrades system performance, it allows larger and multiple processes to run concurrently. Because of this, swapping is also known as **memory compaction.**

The CPU scheduler determines which processes are swapped in and which are swapped out.

In a priority-based scheduling system, high-priority processes are loaded by swapping out low-priority ones. Once the high-priority process finishes, the low-priority process is swapped back to resume execution.



Swapping has been subdivided into two concepts: **swap-in and swap-out**.

- **Swap-out** is a technique for moving a process from **RAM** to the **hard disc**.
- **Swap-in** is a method of transferring a program from a **hard disc to main memory, or RAM.**

Memory Partitioning

Memory partitioning is a technique used in operating systems to divide the computer's main memory into distinct sections, or partitions, to manage multiple processes efficiently. The two primary types of memory partitioning are **fixed partitioning** and **variable partitioning**.

What is Fixed Partitioning?

Fixed Partitioning is a contiguous memory management technique in which the main memory is divided into fixed sized partitions which can be of equal or unequal size. Whenever we have to allocate a process memory then a free partition that is big enough to hold the process is found. Then the memory is allocated to the process. If there is no free space available then the process waits in the queue to be allocated memory. It is one of the oldest memory management technique which is easy to implement.

Advantages of Fixed Partitioning

- **Simplicity:** Since partitions are already defined, it is easy to assign memory to a process especially when it is designed that way.
- **Ease of Implementation:** This makes partitioning method very easy to implement especially since it only requires partitioning of a few segments.

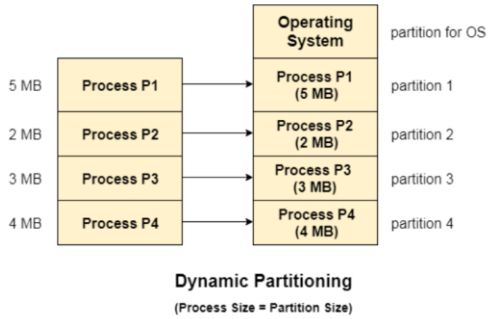
Disadvantages of Fixed Partitioning

- **Internal Fragmentation:** If a process is not able to fill a partition it uses then there is space left in that partition which is not utilized.
- **Inefficient Memory Utilization:** The size of partitions is fixed and therefore large processes might occupy more space while small processes occupy inadequate large space.
- **Limited Process Size:** Quantities greater than the size of the largest partition cannot be performed; this decreases flexibility.

What is Variable Partitioning?

Variable partitioning is a memory management technique in operating systems where the main memory is divided into variable-sized partitions to allocate memory dynamically based on the needs of processes. Unlike fixed partitioning, where partitions have a predefined size, variable partitioning adapts to the actual memory requirements of a process, leading to better utilization of memory.

The first partition is reserved for the operating system



Advantages:

1. **Dynamic Allocation**: Memory is allocated dynamically as processes request it, using just enough memory to meet their requirements.
2. **No Fixed Sizes**: Partitions are created based on the size of the processes, eliminating wasted space caused by unused memory in fixed partitions.
3. **Efficient Memory Use**: Since partition sizes vary, memory utilization is improved, reducing internal fragmentation.

Disadvantages:

1. **External Fragmentation**: Over time, free memory may get divided into scattered blocks, making it hard to allocate large contiguous memory for processes.
2. **Compaction Needed**: To counter external fragmentation, memory compaction may be required, which rearranges memory to combine free spaces.

Memory Allocation Strategies

Memory allocation is a critical task in modern operating systems, and one of the most commonly used techniques is contiguous memory allocation. Contiguous memory allocation involves allocating memory to processes in contiguous blocks, where the starting address of each block is adjacent to the previous one.

1.First Fit

The first-fit algorithm searches for the first free partition that is large enough to accommodate the process. The operating system starts searching from the beginning of the memory and allocates the first free partition that is large enough to fit the process. For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 15 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory. The operating system starts searching from the beginning and finds the first free partition of 20 KB. It allocates the process to that partition and keeps the remaining 2 KB as free memory.

2.Best Fit

The best-fit algorithm searches for the smallest free partition that is large enough to accommodate the process. The operating system searches the entire memory and selects the free partition that is closest in size to the process.

For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 15 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory. The operating system searches for the smallest free partition that is larger than 18 KB, and it finds the partition of 20 KB. It allocates the process to that partition and keeps the remaining 2 KB as free memory.

3.Worst Fit

The worst-fit algorithm searches for the largest free partition and allocates the process to it. This algorithm is designed to leave the largest possible free partition for future use.

For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 15 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory. The operating system searches for the largest free partition, which is 30 KB. It allocates the process to that partition and keeps the remaining 12 KB as free memory.

Pros and Cons

- First Fit is fast and simple to implement, making it the most commonly used algorithm. However, it can suffer from external fragmentation, where small free partitions are left between allocated partitions.
- Best Fit reduces external fragmentation by allocating processes to the smallest free partition, but it requires more time to search for the appropriate partition.
- Worst Fit reduces external fragmentation by leaving the largest free partition, but it can lead to inefficient use of memory.

Fragmentation

Fragmentation is defined as when the process is loaded and removed after execution from memory, it creates a small free hole. These holes cannot be assigned to new processes because holes are not combined or do not fulfill the memory requirement of the process. To enable multiprogramming, we need to minimize memory waste and fragmentation issues.

In the operating systems two types of fragmentation:

1. Internal fragmentation:

Internal fragmentation occurs when memory blocks are allocated to the process more than their requested size. Due to this some unused space is left over and creating an internal fragmentation problem.

Example: Suppose there is a fixed partitioning used for memory allocation and the different sizes of blocks 3MB, 6MB, and 7MB space in memory. Now a new process p4 of size 2MB comes and demands a block of memory. It gets a memory block of

3MB but 1MB block of memory is a waste, and it cannot be allocated to other processes too. This is called **internal fragmentation.**

2. External fragmentation:

In External Fragmentation, we have a free memory block, but we cannot assign it to a process because blocks are not contiguous.

Example: Suppose (consider the above example) three processes p1, p2, and p3 come with sizes 2MB, 4MB, and 7MB respectively. Now they get memory blocks of size 3MB, 6MB, and 7MB allocated respectively. After allocating the process p1 process and the p2 process left 1MB and 2MB. Suppose a new process p4 comes and demands a 3MB block of memory, which is available, but we cannot assign it because free memory space is not contiguous. This is called **external fragmentation.**

First-fit and best-fit memory allocation face external fragmentation. Solutions include **compaction**, which merges free memory into a single block, and allowing **noncontiguous memory allocation**, enabling processes to use available memory efficiently.

Paging in Operating System

Paging is a technique that divides memory into fixed-sized blocks. The process of retrieving processes in the form of pages from the secondary storage into the main memory is known as **paging**.

The main memory is divided into blocks known as Frames, and the logical memory is divided into blocks known as Pages.

The mapping between logical pages and physical page frames is maintained by the page table, which is used by the memory management unit to translate logical addresses into physical addresses.

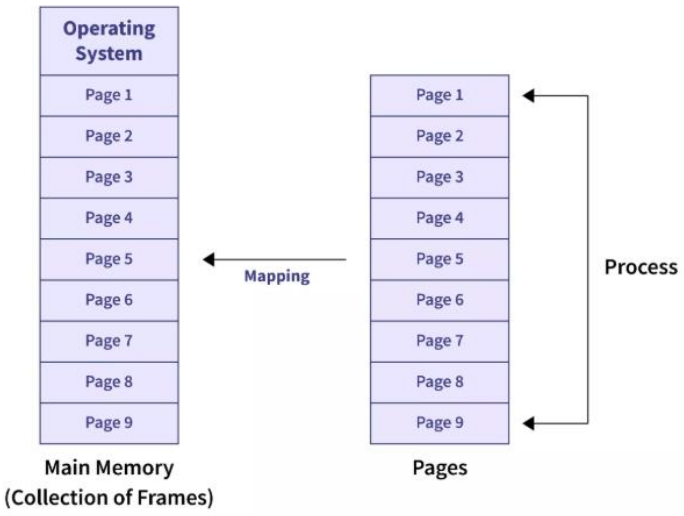
The page table maps each logical page number to a physical page frame number. To speed up address translation and reduce overhead, a special hardware cache memory called the Translation Lookaside Buffer (TLB) is used.

This concept of Paging in OS includes dividing each process into pages of equal size, while the main memory is divided into frames of fixed size.

When a process is loaded into memory, its pages are stored in available frames, ensuring efficient utilization of memory. It is essential that the sizes of pages and frames match for accurate mapping and to avoid memory waste.

Paging allows processes to be stored at non-contiguous locations in memory, which helps prevent external fragmentation. At the same time, the page table ensures that processes remain isolated from each other, maintaining memory protection.

While paging offers efficient memory use and better process management, it has drawbacks, such as the overhead of maintaining the page table and the performance cost of TLB misses.



If a process has n pages in the secondary memory, then there must be n frames available in the main memory for mapping.

Page Faults: When a process accesses a page that is not in physical memory, a page fault occurs. The Memory Management Unit (MMU) triggers an interrupt, prompting the operating system to load the missing page from secondary storage (like a hard disk) into a free frame in RAM.

Page Replacement: If all frames in physical memory are in use, the operating system must choose a page to evict. This process, known as page replacement, is often managed using algorithms like the Least Recently Used (LRU) or the Second Chance algorithm.

Advantages of Paging in OS include

- Efficient memory management by storing pages non-contiguously
- solving external fragmentation
- simplifying page allocation and swapping between fixed-size frames.

Disadvantages of Paging in OS include

- Potential internal fragmentation
- Extra memory usage for page tables
- High access time without TLB